GitNotes

Personal Concise Notes

My Personal Concise Notes about how to use Git and GitKraken with GitHub and BitBucket.

Quick Start

```
                "Git is a database.
                Commit is a record."
```

```
git status        # current status of the git directory
git config --list # current config parameters
git log           # show log
        --graph --oneline --all --decorate

git config
        --list
        --global user.name <"full name">
        --global user.email <"email">
        --global core.editor <"command">
                  # eg.: "gedit --new-window --wait"

git init          # create a local repository
git add <file>    # stage <file> for commit and track changes to it
git commit        # commits staged files to local repository
git restore --staged <file> # unstage <file>
git restore --staged .      # unstage all staged files

git rm <list of files>      # delete tracked files
git mv <list of files> <destination>  # move/rename tracked file(s)

git remote add origin <url> # add url of existing remote repository
git push origin master      # push <branch> to <remote>

git branch <branchname>     # (create and) switch to branch
```

Table of Contents

Initializing a Remote Repository on GitHub

We used to be able to create a remote repository from the git command line, but this is no longer allowed. Now, you must either create the repository manually using the web interface, or by using GitHub CLI which accesses GitHub's remote API. This chapter explains how to initialize a new remote Git repository on GitHub from the terminal command line on your development machine.

The GitHub CLI Interface

Token / SSH Keys

You should already be in possession of the token or SSH keys necessary to access either your GitHub account or the repository on which you are working.

Install gh

Install gh.

For example, in Windows you could use PowerShell or download the MSI, and in Linux you can use apt/dpkg, yum/rpm, etc.

Authorize

This is how you authorize gh to access GitHub from the command line using gh:

```
gh auth login
→ github.com
→ https or ssh
    - https
        → paste token
            - token must have minimum scope
                repo
                read:org
                workflow
            - also useful:
                delete_repo
    - ssh
        - you're done
```

Create Remote Repo From Command Line

Basically:

gh repo create <reponame> --public --source=. --remote=origin

If you do not already have your local repository initialized, the full procedure would be:

```
mkdir <reponame>
cd <reponame>
git init
git branch -M master
echo "# <reponame>" >> README.md
git add README.md
git commit -m "Initial commit."
gh auth login
gh repo create <reponame> --public --source=. --remote=origi
# git remote add origin https://github.com/<user>/<reponame>
git push -u origin master
```

Pushing An Existing Local Repo To An Existing Remote Repo

```
git remote add origin https://github.com/<user>/<repo>.git
git branch -M master
git push -u origin master
```

* <user> is your username on GitHub; <repo> is the full reponame on GitHub.

GitHub Markdown Language for .md Files

Some of this is apparently inaccurate. At the moment, GitHub's ***Basic writing and formatting syntax*** page which documents GitHub-Flavoured Markdown (GFM) is located at:

https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax

Comment	<!-- wrap text →
Heading	# h1, ## h2, ### h3, etc.
Issue	#issue-number or #pull-request (to automatically link)
Bold	**bold text**
Italic	*italicised text* or _italicised text_
Bold-Italic	***bold italicised text***
Blockquote	> quoted text
Ordered List	1. first item; 2. second item, etc.
Unordered List	- first item or * first item, etc.
Inline Code	`code`
Code Block	``` fenced code block ```
Horizontal Rule	~~~ or ***
Link	[anchor](https://url.tld)
Image	![alt text](image.jpg)
Mention	@username (to automatically link)
Footnotes	Sentence with a footnote.[^1] [^1]: This is the footnote.

Headings can have custom ids:

My Great Heading {#custom-id}

Tables:

```
|Heading1|Heading2|
|---|---|
|Header|Title|
|Paragraph|Text|

minimum width:    ----- (number of dashes)
left justify:     :---
right justify     ---:
fully justify     :---:
```

Definition List

term

: definition

Strikethrough

~~~struckthrough text~~~

Task List

- [x] completed task

- [ ] incomplete task

Subscript

H~2~O

Superscript

x^2

Callout / Alert

> [!NOTE] / [!TIP] / [!IMPORTANT] / [!WARNING] / [!CAUTION]

> useful information to know

Diagrams and Math

GitHub supports creating diagrams using Mermaid syntax.

Collapsed Section

<details>

<summary>Place summary here.</summary>

text, images, code blocks, headings, etc.
~~~

Workflow

This chapter documents the chain of events a typical user would follow when working on a project using Git.

Clone

Generally, when collaborating with others, you will clone an entire repository, including all branches and the entire history.

You can, however, begin by cloning an existing single branch. That branch can be master, main, dev or whatever. If you are creating a new branch you must at least clone the branch from which the new branch will be derived.

```
git clone --branch master --single-branch <repo_url>
```

```
--branch          # which branch to checkout after cloning
--single-branch   # only clone the history and files
                    related to the specified branch
git fetch …       # additional branches if needed
```

And even if you are only cloning a single branch, you're usually downloading all the current files in the project anyway.

Create A New Feature Branch

Create a new branch from master or develop dedicated to the feature, bugfix, chapter, etc. The branch name should be in the format:

```
        feature/<feature>

        hotfix/<issue`ID>

        bugfix/issueID>

        chapter/<chaptername>
```

```
git branch chapter/<branchname>
```

Check out the branch:

```
git checkout chapter/<branchname>
```

Provide a detailed description for the branch:

```
git branch --edit-description
```

To read the description for a branch:

```
git config branch.<branchname>.description
```

Commit and Push Regularly

As it develops, commit the code regularly to the local repository. Commit only *working* code, structural changes, and *successful* refactoring; generally, you don't want to commit changes or ideas which are incomplete.

Regularly push the branch on which you are working – ***not*** the entire repo! ***Not*** the branch already merged into main/master/develop! While it's true that you will merge the branch into the master development branch to test features, bug fixes, etc., you should only ever push the branch on which you are working to the remote repository.

Regularly pushed code serves as a backup and allows for team collaboration.

Open A Merge Request

Sometimes called a "pull request". You probably wouldn't actually perform this step if you are the sole developer of the project.

When a feature is complete, tested locally, and the branch has been pushed to the repository, open a merge request on the remote repository. This informs the maintainers of the project that you have completed your feature or bugfix and are requesting that it be merged into the master branch.

If your merge request is rejected, and assuming that you are part of a team which is actively developing the project, find out why the pull was rejected, fix the problem, and resubmit.

You typically do this through the website / repository interface, or if using GitHub you can use the "gh" command line interface:

```
        gh pr create
```

Review and Test

When a Merge Request is received, the changes are reviewed by teammates. Automated tests are usually run, at this point, to ensure code quality. If the code requires further development or polishing, send it back to the programmer or another teammate.

Merge Into Master/Main/Dev

When the code is complete, working, and clean, merge it into the appropriate branch of the repository. This can be done directly on the GitHub or BitBucket platforms and is, in fact, the preferred way to do it to ensure that the master/main/dev/stable branches do not change during this update.

Keep or Delete the Branch

Keeping or deleting merged branches is a topic of perennial discussion.

If you anticipate further development on a feature you can keep the branch, otherwise you should probably just delete it. In a well-organized project there is very dubious gain in holding on to multiple branches of history of past work.

To delete a local branch:

```
git branch -d <branchname>
```

Note that remote branches must be explicitly deleted. They implicitly deleted as the result of a push. To delete a remote branch:

```
git push origin --delete <branch_name>
```

To restore a deleted branch use:

```
git reflog
```

to find the entry related to the delete branch, then use:

```
git checkout -b <branch_name> <sha_hash_from_reflog>
```

to restore the branch locally. Then, at some point, use:

```
git push origin <branch_name>
```

to also restore the branch remotely.

Alternatively, a teammate who still has a copy of the deleted branch can push it up to the repo and you can pull it back down.

GitHub also has a temporary “Restore Branch” button which appears in the list of branches in the web interface.

Typical Branches

Note that this section is a work in development. My notes have a number of inconsistencies when it comes to branch derivations, usage, and merges, and I’ve tried to highlight these inconsistencies for future clarification or correction.

This list of typical branch names and their uses should serve as a guide. Actual branch names for your project are up to you. As Git is a versioning platform initially intended for coders, the branch names and descriptions are taken from that culture.

main / master: All projects have, or should have, a master branch which serves as a springboard for all other long-running branches all other branches; it is the branch from which all long-running branches are derived. Nothing happens here. For large and/or complex projects with significant collaboration, this is just the skeleton on which the infrastructure is hung. The default name for this branch these days is “main”; the former default master branch name of “master” has been deprecated. Nothing happens here? What would be the purpose of having a master branch at all which apparently does nothing?

stable: Derived from the master branch and should only receive merges from “release/*” and “hotfix/*” branches. How does “stable” differ from “develop”? When should “develop” be merged into “stable”

hotfix/* / hot/*: Derived from the master branch and used for the fixing of critical bugs or security vulnerabilities in a live release without waiting for the next scheduled update. The format of this branch name should be “hotfix/<issueID>”. This branch is merged back into “stable”.

dev / develop: Derived from main, this is the primary development branch. This branch should only receive merges from “feature/*” and “bug/*” branches. Why have a separate develop branch instead of doing all your development on the master branch? How does “develop” differ from “stable”, and when does “develop” get merged into “stable”.

feature: Derived from the HEAD of “develop”. Generally used for adding or removing features and other changes. The format of this branch should be “feature/<feature_name>”. It gets merged back into “develop”. From the HEAD of “develop”? If “develop” is not a dynamic branch, why not derive it from “release/*” instead of “develop”?

bugfix/* / bug/*: Derived from the HEAD of “develop”. This branch is used for fixing low-priority bugs which are scheduled to be fixed in the next release cycle. The format of this branch should be “bugfix/<bugID>”. It gets merged back into “develop”. From the HEAD of “develop”? If “develop” is not a dynamic branch, why not derive it from “release/*” instead of “develop”?

release/*: Derived from “develop”. The format of this branch should be “release/<version>”. Other than having a version number, how does “release/*” differ from “stable”?

nightly build: Also not covered here is the concept of the “nightly” or “unstable” build.

Oh yeah: There should also be a branch for documentation. Maybe that is what gets rolled up into “release/*” rather than into “develop” or “stable”.

** Also, if working on a book, the use of “revision/*” rather than “release/*” may be more appropriate, as is “chapter/*” rather than “feature/*”. Additional branches such as “errata”, often published as a separate document, may also be appropriate.

THERE SHOULD BE ONE BLANK LINE AFTER/BETWEEN EACH SECTION.

BitBucket

Don't use BitBucket. BitBucket was purchased by Atlassian, so you are really creating an Atlassian account to which you are attaching a free BitBucket subscription. Under this new regime I found repositories and workspaces to be difficult to manage and delete. You cannot delete your Atlassian account while you are subscribed to BitBucket, and you cannot unsubscribe from BitBucket because it is a free subscription.

I also spent way too much time trying to get a push to work from the command line, much less trying to integrate BitBucket with GitKraken. It's an inconvenience up with which one need not put when one has access to GitHub.

Index